Actividad Práctica en Parejas: Primer flujo de trabajo colaborativo

Objetivo: Recorrer el ciclo completo de trabajo con Git: clonar, preparar, confirmar, publicar y traer cambios. También provocar, identificar y resolver un conflicto.

Antes de empezar

Cada integrante debe tener el entorno preparado según el documento de pre clase (Git y GitHub CLI instalados, git config hecho y gh auth login ejecutado), y actividad.zip descargado desde Google Classroom.

Definan los roles y anótenlos: Estudiante A (dueño del repositorio) y Estudiante B (colaborador).

Regla de la actividad: después de cada comando que modifique algo, ejecutar git status y leer la respuesta antes de seguir. La mitad del aprendizaje de hoy está en esa lectura.

Si la pareja tiene una sola computadora: clonen el repositorio dos veces, en dos carpetas distintas (poo-A y poo-B), y trabaje cada uno en la suya alternando el turno en la terminal. El flujo y el conflicto se reproducen igual.

Paso 0: Verificación del entorno

Cada integrante, en su terminal:

```
git --version
git config --global --list
gh auth status
```

Los tres deben responder correctamente. En la salida del segundo tienen que aparecer user.name, user.email y **pull.rebase=false**: sin esa última línea, el paso 5 se corta con un error. Si algo falla, resuélvanlo ahora.

Paso 1: Crear y compartir el repositorio

Estudiante A, desde el navegador:

1. En https://github.com/new, crear un repositorio:
   - ➢ Repository name: poo2026-git-apellidoA-apellidoB (sin espacios ni acentos).
   - ➢ Visibility: *Public*.
   - ➢ Tildar Add a README file y elegir la plantilla de .gitignore para Java.
2. Copiar la URL del repositorio (botón verde Code, pestaña HTTPS).
3. Ir a Settings → Collaborators → Add people e invitar al Estudiante B con su usuario de GitHub.

Estudiante B: aceptar la invitación (llega por correo y también aparece al entrar a la URL del repositorio). Entrar a la URL y ver el botón verde Code. Si dice *pending invitation*, la invitación todavía no fue aceptada.

Paso 2: Clonar

Los dos, cada uno en su máquina. Primero ubicarse en la carpeta donde quieran guardar el proyecto (cd ~/Documentos, por ejemplo) y después:

```
git clone <url-del-repositorio>
cd <nombre-del-repositorio>
git status
git log --oneline
```

git status debe responder On branch main y nothing to commit, working tree clean. git log muestra un único commit, el que creó GitHub al inicializar el repositorio.

Paso 3: El primer commit

Estudiante A:

1. Descomprimir actividad.zip dentro de la carpeta del repositorio. Quedan la carpeta src (con App.java), COMANDOS.md, un README.md nuevo y un .gitignore ampliado: los dos últimos reemplazan a los que había generado GitHub. El .gitignore del zip agrega las carpetas del IDE y de compilación, que la plantilla de GitHub no incluye.
2. Completar en el README.md los nombres de los dos integrantes.
3. Ver qué detectó Git, prepararlo y confirmarlo:

```
git status
git add .
git status
git commit -m "Agrega estructura base del proyecto"
git status
```

Comparen los tres git status: primero los archivos aparecen como *untracked* o *modified*; después de git add pasan a *Changes to be committed*; después del commit aparece Your branch is ahead of 'origin/main' by 1 commit. Esa última línea dice que el trabajo todavía no salió de esta computadora.

4. Publicarlo y avisarle al compañero:

```
git push
git status
```

Estudiante B, cuando A avisa:

```
git pull
git log --oneline
```

En este punto los dos tienen los mismos archivos y ven en el historial el commit del Estudiante A, con su nombre y su mensaje.

Paso 4: provocar el conflicto

Este paso se hace sin sincronizar entre ustedes: cada uno trabaja como si el otro no existiera. Los dos, al mismo tiempo y sin hacer git pull:

1. Abrir src/App.java y modificar la misma línea, la del System.out.println, con un saludo propio:

- ➢ Estudiante A: System.out.println("Hola, soy <NOMBRE> y esto es POO 2026");
- ➢ Estudiante B: System.out.println("Buenas, soy <NOMBRE> y arranco POO");

2. Guardar y confirmar el cambio:

```
git diff
git add src/App.java
git commit -m "Personaliza el saludo de A"
```

Ahora sí, por turnos:

3. Estudiante A ejecuta git push. Funciona.
4. Estudiante B ejecuta git push. Falla:

```
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/...'
hint: Updates were rejected because the remote contains work that you do
not
hint: have locally.
```

Anoten la respuesta antes de seguir: ¿por qué fue rechazado el push de B, si su commit estaba bien hecho? ¿Dónde está guardado ese commit en este momento?

Paso 5: Resolver el conflicto

Resuélvanlo entre los dos, mirando la misma pantalla.

1. El Estudiante B hace lo que sugiere el mensaje anterior:

git pull

Git responde:

```
Auto-merging src/App.java
 CONFLICT (content): Merge conflict in src/App.java
 Automatic merge failed; fix conflicts and then commit the result.
```

2. Preguntarle a Git dónde está parado:

git status

Ahora avisa que las dos historias divergieron (Your branch and 'origin/main' have diverged) y marca el archivo: You have unmerged paths → both modified: src/App.java.

3. Abrir el archivo y leer los marcadores:

```
<<<<<<< HEAD
        System.out.println("Buenas, soy <NOMBRE> y arranco POO");
=======
        System.out.println("Hola, soy <NOMBRE> y esto es POO 2026");
>>>>>>> b03a76c44c3a85c58136baceb749104e1fa0bec3
```

Entre <<<<<<< HEAD y ======= está la versión local (la de B); entre ======= y >>>>>>>, la que llegó del remoto (la de A). Ese código largo del final es el identificador del commit que llegó: en su pantalla va a ser otro.

4. Editar el archivo para que queden las dos líneas de saludo, una debajo de la otra, y borrar los tres marcadores. Tiene que quedar código Java válido. Guardar.

5. Avisarle a Git que está resuelto y cerrar la fusión:

```
git add src/App.java
git status
git commit -m "Resuelve el conflicto conservando los saludos de A y B"
git push
```

6. **Estudiante A** trae la versión final y mira la forma del historial:

```
git pull
git log --oneline --graph --all
```

En el grafo se ven las dos líneas de trabajo separándose y volviendo a unirse.

Verificación final

- ☐ src/App.java compila y muestra los dos saludos, en las dos máquinas.
- ☐ No quedó ningún <<<<<<<, ======= ni >>>>>>> en el archivo.
- ☐ El historial tiene commits de los dos integrantes (git log --format="%h %an %s").
- ☐ Existe el commit de fusión.
- ☐ git status dice working tree clean y up to date with 'origin/main'.
- ☐ En el repositorio no hay archivos .class ni carpetas bin/.

Si algo falla:

Mensaje	Qué hacer
fatal: not a git repository	La terminal no está dentro de la carpeta del repositorio: entrar con cd.
Authentication failed o pide contraseña	Ejecutar gh auth login y repetir el comando.
! [rejected] ... (fetch first)	Es lo esperado en la Fase 4: hacer git pull y seguir con la Fase 5.
error: Your local changes would be overwritten	Hay cambios sin confirmar: git add y git commit antes del pull.
Se abrió un editor de terminal del que no se puede salir	Escribir :q y Enter.
Se rompió todo	Borrar la carpeta local y volver a clonar: el historial remoto no se pierde.

Para la puesta en común

1. ¿En qué momento exacto el trabajo dejó de estar solo en la computadora de cada uno?
2. ¿Qué diferencia hay entre git add y git commit? ¿Para qué sirve que sean dos pasos?
3. ¿Qué habría pasado si el Estudiante B hacía git pull antes de empezar a editar, en el paso 4?